KeelNet: An Extended Summary of Controlled Abstention for Evidence-Grounded Question Answering

Naufal M. Kamaruddin Ideun Jeong Author 3

March 28, 2026

Abstract

KeelNet studies a narrow version of hallucination control: evidence-grounded extractive question answering with an explicit answer-or-ABSTAIN decision. Using SQuAD 2.0 [3], the project compares a staged modular pipeline against several direct-learning alternatives. The modular path establishes four useful results: abstention supervision substantially reduces unsupported answers, support verification is learnable, temperature scaling improves both QA and support calibration, and a calibrated controller can satisfy the unsupported-answer budget on validation. The direct-learning and hybrid path establishes four useful negative results: a support-constrained learner recovers answerable-question quality but becomes too permissive, an adaptive revision changes the trade-off only weakly, and a risk-budgeted action learner satisfies validation budgets without carrying that safety gain to the development split, while the cleaned Stage 8.2 action learner repeats the same pattern on a truly untouched test split. The final comparison shows that Stage 5–8.2 recover higher answerable F_1, but the strongest executed overall groundedness trade-off still comes from Stage 4 fixed control, and no executed checkpoint reaches the nominal 20% unsupported-answer budget on the development frontier.

1 Problem and Goal

KeelNet addresses evidence-grounded QA with a constrained action space:

$$f(q, e) \rightarrow \{\text{answer span}, \textsc{abstain}\}, \tag{1}$$

where q is a question and e is a single evidence passage. The goal is not to maximize answer extraction alone, but to improve the final decision trade-off among answer quality, unsupported answering, and abstention.

This framing is intentionally narrower than open-domain hallucination control. The setup excludes retrieval, multi-hop reasoning, and long-form generation so that the central question is clean: can a QA system learn when to answer and when to refuse, while staying tied to the supplied evidence? That question is closely related to selective prediction and calibration [1, 2], but here it is instantiated in a single-passage extractive QA setting with explicit unsupported-answer tracking.

2 Stage Design

The project proceeds through a staged comparison rather than one monolithic method.

1. **Stage 1** compares an answer-only baseline against an abstain-aware model.
2. **Stage 2** adds a support verifier over question, passage, and predicted answer.
3. **Stage 2.5** retrains the verifier with QA-derived hard negatives.
4. **Stage 3** calibrates the QA decision score and support score with scalar temperature scaling.
5. **Stage 4** applies a fixed controller over calibrated QA and support signals under an unsupported-answer budget.
6. **Stage 5** replaces the modular pipeline with a jointly trained support-constrained learner.
7. **Stage 6** adds an adaptive balancing layer on top of Stage 5.
8. **Stage 7** treats candidate answers and ABSTAIN as a shared action space with separate utility and risk signals.
9. **Stage 8** restores calibrated Stage 4 control on top of the frozen Stage 5 answer engine.
10. **Stage 8.2** keeps the frozen Stage 5 answer engine but injects calibrated Stage 4 support into the Stage 7 action learner and reports its final result on a clean final split.

The key experimental question is whether changing the learning objective beats post-hoc modular control under matched data, backbone, and evaluation conditions.

3 Experimental Protocol

All reported results use SQuAD 2.0 with deterministic train, validation, and development splits. The core evaluation metrics are answerable F_1, decision-aware overall F_1, unsupported-answer rate, and abstain F_1. Support-aware stages also report answer rate and supported-answer rate among answered predictions.

The QA backbone is DistilBERT, and later stages keep the same core encoder family while changing decision logic rather than scaling model size. The modular path was executed through Stage 4, and the direct-learning path was executed through Stage 8.2.

Final chosen learned architecture. The paper's final chosen learned architecture is Stage 8.2. In deep-learning terms, it freezes the Stage 5 DistilBERT answer engine as a candidate generator, builds a small top-K answer set, augments each candidate with calibrated Stage 4 support features, and feeds those candidate vectors into a shallow MLP action network with separate utility, risk, and abstain heads. The final score for each answer is a risk-penalized utility logit, while the abstain action is computed from pooled candidate states. This makes Stage 8.2 the cleanest executed support-aware action model in the project, even though Stage 4 remains the strongest overall operating point. Figure 1 shows that architecture as a compact flowchart.

4 Final Learned Architecture

Stage 8.2 is best understood as a compact decision network on top of a frozen QA proposal model. The Stage 5 DistilBERT checkpoint produces a top-K candidate set $\{a_1, \ldots, a_K\}$, and the final controller adds the explicit abstain action $\varnothing$. Each candidate is described by a 12-dimensional feature vector containing span quality, calibrated support, abstention, rank, and overlap cues:

$$x_k = [s_{\text{span}}, \Delta_{\text{best}}, \Delta_{\text{next}}, p_{\text{keep}}, p_{\text{sup}}^{\text{cal}}, p_{\varnothing}, p_{\text{stage6}}, \ell, r, o, \delta, u]. \tag{2}$$

The final controller then applies a shallow MLP:

$$h_k = \text{MLP}(x_k), \tag{3}$$

$$u_k = w_u^\top h_k + b_u, \tag{4}$$

$$r_k = \sigma(w_r^\top h_k + b_r), \tag{5}$$

where u_k is the utility logit for answering with candidate a_k and r_k is the predicted unsupported-answer risk. Abstention is not a thresholded afterthought. The model pools the candidate states with mean and max pooling and predicts a separate abstain logit:

$$u_{\varnothing} = f_{\text{abs}}([\text{mean}(h_1, \ldots, h_K); \max(h_1, \ldots, h_K)]). \tag{6}$$

The final action scores are

$$s_k = u_k - \lambda_{\text{u}} r_k, \qquad s_{\varnothing} = u_{\varnothing} - \lambda_{\text{a}}, \tag{7}$$

followed by softmax over $[s_1, \ldots, s_K, s_{\varnothing}]$. At inference time, a hard calibrated-support shield can still suppress candidates whose Stage 4 support probability falls below the selected floor. So the architecture is hybrid in the deepest sense: transformer QA proposals, calibrated verifier support, and a small learned action layer with explicit utility, risk, and abstain heads.

5 Main Findings

1. Abstention supervision works, but not for free. Stage 1 changes the baseline from pathological always-answer behavior into a meaningful abstention policy. Unsupported-answer rate drops from 100.00% to 27.97%, and overall F_1 rises from 42.35 to 69.40. The cost is substantial: answerable F_1 falls from 84.82 to 66.76. So abstention is learnable, but it introduces a real usefulness-versus-caution trade-off.

2. Support is learnable, but the verifier remains too permissive. Stage 2 shows that support verification is not random noise. The verifier reaches strong support F_1, but the downstream gated QA system changes end-to-end behavior only marginally. Stage 2.5 sharpens this diagnosis. Hard negatives make the verifier slightly less permissive, lowering dev unsupported-answer rate from 27.57% to 26.95%, but they also lower answerable F_1 and overall F_1. This means the verifier-side bottleneck is real, but not removable by a simple data repair alone.

3. Calibration improves the control signals, especially for QA. Stage 3 confirms that both QA and support scores are strongly overconfident. QA expected calibration error falls from 0.2456 to 0.0753 after temperature scaling, while support ECE falls from 0.3326 to 0.1708. This matters because later controller stages depend on score quality rather than only raw accuracy. However, support remains the weaker signal even after calibration, which foreshadows the limits of later controller and learner stages.

4. The first calibrated controller is validation-feasible but development-flat. Stage 4 satisfies the nominal 20% unsupported-answer budget on validation with 14.30% unsupported-answer rate and 72.43 overall F_1. That is a meaningful proof that the calibrated scores can support a constrained selection policy. But on development, Stage 4 is effectively a null result relative to Stage 2: 69.41 overall F_1 versus 69.42, and 27.59% unsupported-answer rate versus 27.57%. So calibration improves score quality, but the first fixed controller still does not leverage those better scores into a better held-out decision rule.

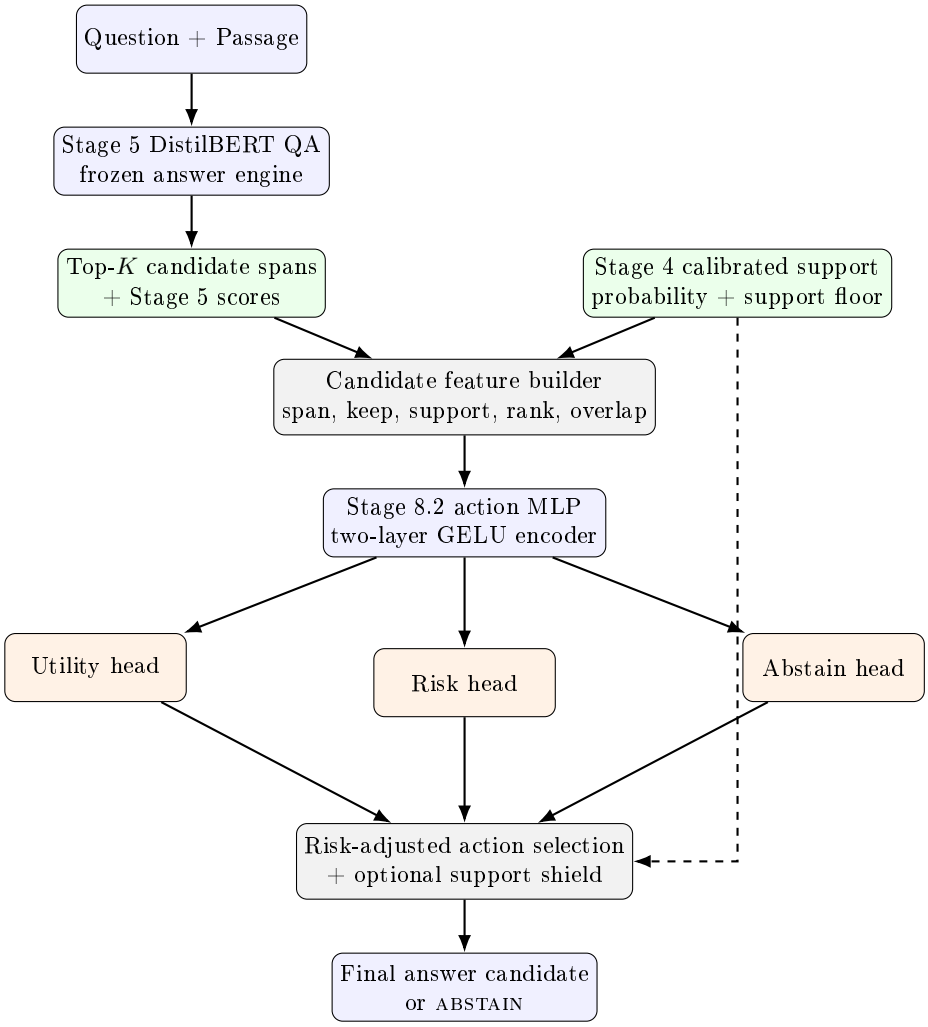

Figure 1: Mermaid-like LaTeX flowchart of the final chosen Stage 8.2 architecture. Frozen Stage 5 QA generates the candidate spans, calibrated Stage 4 support enriches and filters them, and a compact action network predicts utility, risk, and abstain scores before the final answer-or-ABSTAIN choice.

5. Direct learning recovers answer quality faster than it recovers groundedness. Stage 5 is the clearest positive signal for the direct-learning path. It raises answerable F_1 to 71.40, the best executed result in the later Stage 1/4/5/6/7 comparison. But it does so by becoming much more permissive: unsupported-answer rate rises to 34.84% and overall F_1 falls to 68.28. This is a useful negative result, because it reveals the actual failure mode of the first support-constrained objective: answer extraction improves faster than unsupported-answer control.

6. The first adaptive, action-based, and hybrid revisions are informative but still not wins. Stage 6 lowers unsupported-answer rate only slightly, from 34.84% to 34.45%, while giving back both answerable F_1 and overall F_1. Stage 7 is more ambitious. It satisfies both unsupported-answer and over-abstention budgets on validation, which is the strongest validation-time safety result in the project. Yet on development it still reaches 34.62% unsupported-answer rate, with 67.66 overall F_1. Stage 8 Hybrid partially recovers groundedness, reaching 68.27 overall F_1 and 31.86% unsupported-answer rate on development, but it still does not recover the Stage 4 modular frontier. The clean-split Stage 8.2 variant sharpens the same diagnosis: it satisfies the 20% unsupported-answer budget on validation with 19.31% unsupported-answer rate, then falls back to 33.73% unsupported-answer rate and 68.16 overall F_1 on the untouched test split. So the current bottleneck is no longer just missing action structure or missing budgets. It is learning a risk signal whose supported-versus-unsupported separation generalizes beyond validation.

7. The final comparison shows two distinct regimes rather than one improving frontier. The aggregated final comparison in Figure 2 makes the

Table 1: Executed checkpoints on the held-out development split. Higher is better for answerable F_1 and overall F_1; lower is better for unsupported-answer rate. Stage 8.2 is discussed separately because its clean rerun reports final results on the untouched test split rather than on development.

System	Answerable F_1	Overall F_1	Unsupported Rate
Stage 1 baseline	84.82	42.35	100.00
Stage 1 abstain-aware	66.76	69.40	27.97
Stage 2 gated QA	66.41	69.42	27.57
Stage 2.5 hard-negative gated QA	65.62	69.34	26.95
Stage 4 fixed controller	66.41	69.41	27.59
Stage 5 support-constrained learner	71.40	68.28	34.84
Stage 6 adaptive learner	69.70	67.62	34.45
Stage 7 action learner	69.95	67.66	34.62
Stage 8 hybrid controller	68.39	68.27	31.86

overall picture especially clear. On the development split, the modular checkpoints cluster around 69.3–69.4 overall F_1 with about 27% unsupported-answer rate. By contrast, Stage 5–8 Hybrid occupy a different region with higher answerable F_1 but lower overall F_1 and roughly 32–35% unsupported-answer rate. The clean-split Stage 8.2 test point lands in the same broad learned-model regime rather than breaking through it. In other words, the current direct-learning family has moved the system into a different operating regime, but not yet to a better groundedness frontier. No executed checkpoint reaches the nominal 20% unsupported-answer budget on the held-out development split.

6 Conclusion

The project has now established several useful positive results and several equally useful negative ones. The positive results are that abstention supervision works, support prediction is viable, score calibration improves the quality of downstream control signals, and a validation-feasible calibrated controller can be selected. The negative results are that verifier repair alone does not remove the modular bottleneck, the first jointly trained learner is too permissive, the first adaptive revision is too weak, and neither the risk-budgeted nor the cleaned hybrid revisions transfer their validation-side safety gains tightly enough to held-out data.

The final architectural lesson is therefore specific. If the goal is the best executed operating point today, Stage 4 fixed control is still the answer. If the goal is the clearest deep-learning formulation of support-aware answer selection, Stage 8.2 is the right final model to carry forward: frozen QA proposals, calibrated support features, and a learned utility-versus-risk action layer with an explicit abstain head.

The strongest current conclusion is therefore diagnostic rather than triumphant: the remaining bottleneck is not whether abstention, support, or calibration are possible at all, but whether the system can learn a risk signal that generalizes tightly enough to control unsupported answering on held-out data. That keeps the main research direction alive, but it also changes the standard for success. Future work should be judged not by whether it merely changes the trade-off, but by whether it improves groundedness without collapsing utility on the same development frontier.

References

[1] Yonatan Geifman and Ran El-Yaniv. Selectivenet: A deep neural network with an integrated reject option. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2151–2159, 2019. URL https://proceedings.mlr.press/v97/geifman19a.html.

[2] Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. How can we know when language models know? on the calibration of language models for question answering. *Transactions of the Association for Computational Linguistics*, 9:962–977, 2021. doi: 10.1162/tacl_a_00407. URL https://aclanthology.org/2021.tacl-1.57/.

[3] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, 2018. doi: 10.18653/v1/P18-2124. URL https://aclanthology.org/P18-2124/.

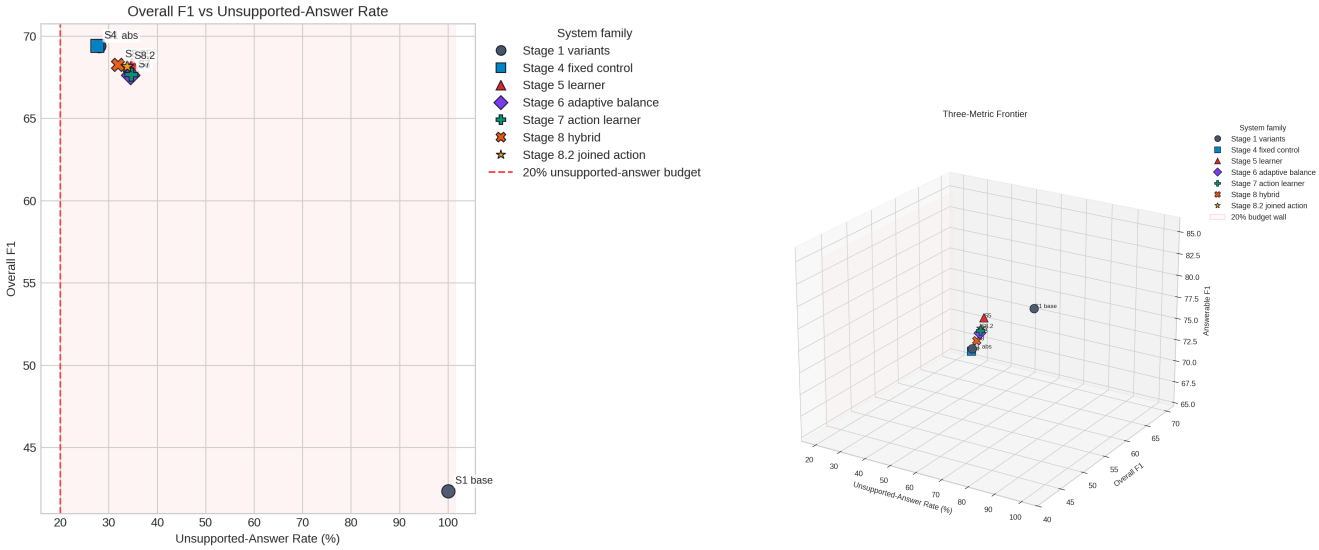

Figure 2: Notebook-generated final comparison frontier. Higher is better on the vertical axis and lower is better on the horizontal axis in the left plot. The main development frontier is still anchored by the modular checkpoints, with Stage 4 as the best executed overall trade-off. The starred Stage 8.2 point is included as a clean-split test reference: it raises answerable F_1 relative to Stage 4, but remains in the same higher-risk regime as the other learned variants. The 3D view on the right makes the same point using unsupported-answer rate, overall F_1, and answerable F_1 jointly: the learned family shifts toward utility, but the safer frontier remains modular.